

Homework3 参考答案

Problem 1

```
int counter = 2;
void handler1(int sig) {
    counter = counter - 1;
    printf("%d", counter); fflush(stdout);
    exit(0);
}
int main() {
    signal(SIGUSR1, handler1);
    printf("%d", counter); fflush(stdout); // 父进程打印 2
    if ((pid = fork()) == 0) { while(1) {}; }
    kill(pid, SIGUSR1); // 给子进程发 SIGUSR1
    waitpid(-1, NULL, 0); // 等子进程结束
    counter = counter + 1;
    printf("%d", counter); // 父进程打印 3
    exit(0);
}
```

父进程先打印 counter=2。fork 后子进程拷贝得到 counter=2 并进入死循环。父进程 kill 子进程，子进程的 handler1 把自己的 counter 减为 1，打印 1 后 exit。子进程对 counter 的修改不影响父进程。父进程 waitpid 回收子进程后，counter 仍是 2，加 1 变 3，打印 3。

由于 2 在 fork 前打印、3 在 waitpid（子进程已打印 1 并结束）之后打印，顺序固定。

输出：213

Problem 2

snooze 程序：命令行读入一个整数 secs，睡眠 secs 秒后终止；睡眠期间按 Ctrl-C（SIGINT）则提前唤醒并打印已睡眠的时间。利用 sleep() 被信号中断后返回“剩余 未睡的秒数”这一特性即可。

```
#include "csapp.h"

void sigint_handler(int sig) {
    return; /* 仅用于中断 sleep，不做其他事 */
}

unsigned int snooze(unsigned int secs) {
    unsigned int rc = sleep(secs); /* 被中断则返回剩余秒数 */
    printf("Slept for %u of %u secs.\n", secs - rc, secs);
    return rc;
}

int main(int argc, char **argv) {
    if (argc != 2) {
```

```

        fprintf(stderr, "usage: %s <secs>\n", argv[0]);
        exit(0);
    }
    if (signal(SIGINT, sigint_handler) == SIG_ERR) {
        fprintf(stderr, "signal error\n");
        exit(1);
    }
    snooze((unsigned int) atoi(argv[1]));
    exit(0);
}

```

要点：用 signal 注册 SIGINT 处理函数，处理函数只需返回即可打断 sleep；sleep 正常睡满返回 0，被信号打断返回剩余秒数，用 secs - rc 得到实际睡眠时间。

Problem 3

任务到达与执行时间：

任务	到达时刻	执行时间
T1	0	8
T2	0	3
T3	0	2
T4	2	10
T5	2	1

周转时间 = 完成时刻 - 到达时刻；响应时间 = 首次开始运行时刻 - 到达时刻。

(1) FCFS

按到达顺序：T1, T2, T3, T4, T5。

任务	开始	完成	周转	响应
T1	0	8	$8 - 0 = 8$	$0 - 0 = 0$
T2	8	11	$11 - 0 = 11$	$8 - 0 = 8$
T3	11	13	$13 - 0 = 13$	$11 - 0 = 11$
T4	13	23	$23 - 2 = 21$	$13 - 2 = 11$
T5	23	24	$24 - 2 = 22$	$23 - 2 = 21$

平均周转 = $(8+11+13+21+22)/5 = 75/5 = 15$ 平均响应 = $(0+8+11+11+21)/5 = 51/5 = 10.2$

(2) RR, 时间片 = 1

约定（按题意）：每一轮里按到达顺序服务，轮转过程中新到的请求加入“本轮”的末尾，已经在本轮跑过一次的任务降到“下一轮”。据此逐个时间片模拟：

时间片： 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23

任务： T1 T2 T3 T4 T5 T1 T2 T3 T4 T1 T2 T4 T1 T4 T1 T4 T1 T4 T1 T4 T1 T4 T4
T4

(第1轮 T1 T2 T3 跑过后, t=2 到达的 T4 T5 接在本轮末尾, 故第1轮是 T1 T2 T3 T4 T5; 之后 T1 等降级到下一轮。)

任务	完成	周转	首次运行	响应
T1	21	$21-0=21$	0	0
T2	11	$11-0=11$	1	1
T3	8	$8-0=8$	2	2
T4	24	$24-2=22$	3	$3-2=1$
T5	5	$5-2=3$	4	$4-2=2$

平均周转 = $(21+11+8+22+3)/5 = 65/5 = 13$ 平均响应 = $(0+1+2+1+2)/5 = 6/5 = 1.2$

(3) 抢占式 SJF (最短剩余时间优先)

t=0 就绪 T1(8), T2(3), T3(2), 选最短 T3, 运行到 t=2 完成 (此刻 T4, T5 到达)。t=2 就绪 T1(8), T2(3), T4(10), T5(1), 选 T5, t=3 完成。t=3 选 T2(3), t=6 完成。t=6 选 T1(8), t=14 完成。t=14 选 T4, t=24 完成。本例中各次选择恰好不需要打断正在运行的任务。

调度顺序：T3, T5, T2, T1, T4

任务	完成	周转	首次运行	响应
T3	2	$2-0=2$	0	0
T5	3	$3-2=1$	2	$2-2=0$
T2	6	$6-0=6$	3	$3-0=3$
T1	14	$14-0=14$	6	$6-0=6$
T4	24	$24-2=22$	14	$14-2=12$

平均周转 = $(14+6+2+22+1)/5 = 45/5 = 9$ 平均响应 = $(6+3+0+12+0)/5 = 21/5 = 4.2$

(4) MLFQ

约定 (按题意)：多级队列，每级时间片都为 1；一个任务用完一个时间片就降一级；一个周期长度为 10，即在 t=10、t=20 进行优先级提升 (boost)，把所有任务放回最高 优先级队列 (同级按任务编号/到达先后排序)；新任务进入最高级队列。逐时间片模拟：

时间片： 0 1 2 3 4 5 6 7 8 9 | 10 11 12 13 14 15 16 17 18 19 | 20 21 22 23
 任务： T1 T2 T3 T4 T5 T1 T2 T3 T4 T1 | T1 T2 T4 T1 T4 T1 T4 T1 T4 T1 | T4 T4
 T4 T4
 boost boost

任务	完成	周转	首次运行	响应
T1	20	$20 - 0 = 20$	0	0
T2	12	$12 - 0 = 12$	1	1
T3	8	$8 - 0 = 8$	2	2
T4	24	$24 - 2 = 22$	3	$3 - 2 = 1$
T5	5	$5 - 2 = 3$	4	$4 - 2 = 2$

平均周转 = $(20+12+8+22+3)/5 = 65/5 = 13$ 平均响应 = $(0+1+2+1+2)/5 = 6/5 = 1.2$

说明：RR 与 MLFQ 的具体调度序列依赖于“新到达请求的入队位置”“boost 时刻与 同级排序”等约定，上面已按题面给出的参数写明所用约定；若课程约定不同，序列与 响应时间会有差异，但平均周转时间一般不变。